NOIP 2026 模拟赛（01）题解

T2 基层干部（girl）

【先把题目读懂】

n 名干部（n 是偶数）要分到甲、乙两个县，每个县正好 $n/2$ 人。

编号 $i \geq 2$ 的干部有一个上级，编号是 $\lfloor i/2 \rfloor$。如果 i 和它的上级分到了同一个县，就会产生 d_i 的均衡度。d_i 可能是负数，也就是说“分到一起”不一定好，有时候反而是负担。

求最大的总均衡度。

先把“上级关系”画成图

每个 $i \geq 2$ 都连一条线到 $\lfloor i/2 \rfloor$。这样一共有 n 个点、$n-1$ 条线，而且从 1 出发能走到所有点、路上不会绕圈，所以它正好是一棵**以 1 为根的二叉树**。

1 的孩子是 2 和 3；2 的孩子是 4 和 5；3 的孩子是 6 和 7……规律很整齐。

于是题目变成：

> 在一棵树上挑出正好 $n/2$ 个点染成甲色（其余是乙色），让“两端同色”的边的权值和最大。

手算一个最小的例子

$n=4$，d_1 没用，$d_2=-3$，$d_3=-3$，$d_4=3$。树长这样：1 的孩子是 2 和 3，2 的孩子是 4。

4 个人分两县、每县 2 人，一共只有 6 种分法，全列出来：

甲县	乙县	边 1 到 2	边 1 到 3	边 2 到 4	总分
1, 2	3, 4	同县，-3	不同，0	不同，0	-3
1, 3	2, 4	不同，0	同县，-3	同县，$+3$	0
1, 4	2, 3	不同，0	不同，0	不同，0	0
2, 3	1, 4	不同，0	不同，0	不同，0	0
2, 4	1, 3	不同，0	同县，-3	同县，$+3$	0
3, 4	1, 2	同县，-3	不同，0	同县，$+3$	0

答案最大是 0。注意看：**没有一个方案能让三条边都算上分**，最多只能拿 0。这就是 d 有负数时题目的味道——不能“能加就加”。

这题难在哪

如果 d 全是正数，那当然是把人尽量凑到一起，问题是现在有负数，你得取舍：某条边加 3，可能逼得另外两条边各扣 3，反而亏。所以必须“整体考虑”，不能一条边一条边贪心。

【分数地图】

子任务	测试点	特殊性质	分值
1	1 到 5	$n \le 20$	10
2	6 到 10	所有 $d_i = 1$	10
3	11 到 20	当 $i \le n/2$ 时 $d_i = 0$	20
4	21 到 30	$n \le 80$	20
5	31 到 50	没有特殊限制	50

n 最大 450。看到这个数就知道：$O(n^3)$ 勉强、$O(n^2)$ 正好、$O(n^2)$ 以上都危险。

这次考试的情况：枚举那一档 15 人拿到，但从 $n \le 80$ 那一档开始骤降到 5 人，满分只有 4 人。门槛非常集中，就卡在“把子树打包成状态”这一步上。还有个值得注意的细节：d 全为 1 那一档也只有 4 人整档通过——性质好看不代表可以贪心，那 10 分照样要按正规做法拿。所以下面做法二那一段请你慢点读，它就是全班 11 个人没迈过去的那道坎。

【做法一：一个人一个人地试（10分）】

$n \le 20$，一共就 20 个人，直接枚举“哪些人去甲县”。

用一个整数 `mask` 表示一种分法：`mask` 的第 $i-1$ 位是 1，就表示第 i 个人去了甲县。这样 0 到 $2^n - 1$ 就把所有分法都过了一遍。

对每种分法，先检查甲县是不是正好 $n/2$ 人，再一条边一条边地加分。

```
#include <bits/stdc++.h>
using namespace std;

int n;
long long d[25];

int main() {
    scanf("%d", &n);
    for (int i = 1; i <= n; i++) {
        scanf("%lld", &d[i]);
    }

    if (n > 20) {
        return 0;                          // 这档暴力只管 n ≤ 20，更大的数据直接放弃
    }
```

```
    long long best = -1e18;                // 先设一个很小的数，后面取最大
                                           // 写成 -1e18，别写 18 个零，多数一个就完了
    for (int mask = 0; mask < (1 << n); mask++) {
        int cnt = 0;                           // 甲县里有几个人
        for (int i = 0; i < n; i++) {
            if ((mask >> i) & 1) {
                cnt++;
            }
        }
        if (cnt != n / 2) {
            continue;                          // 不是正好一半，这个方案不算
        }

        long long sum = 0;
        for (int i = 2; i <= n; i++) {
            int son = (mask >> (i - 1)) & 1;       // i 号在不在甲县
            int parent = (mask >> (i / 2 - 1)) & 1;  // i 的上级在不在甲县
            if (son == parent) {
                sum += d[i];                    // 父子同县，这条边才有贡献
            }
        }
        if (sum > best) {
            best = sum;
        }
    }
    printf("%lld\n", best);
    return 0;
}
```

`(mask >> (i - 1)) & 1` 是取出第 i 位的值，这个写法要练熟，以后枚举子集天天用。

这一档够不够

$2^{20} \approx 100$ 万种分法，每种扫 20 条边，两千万次，一秒内跑完，这 5 个点稳过。

但 $n = 450$ 时 2^{450} 是个天文数字，完全没戏。所以必须换思路，不能“枚举所有分法”。

【做法二：一条边的分，只看它两个端点】

先想一个很朴素的问题：**一条边到底算不算分，由谁决定？**

答案很干脆：只由这条边的两个端点决定，跟树上其他任何点都没关系。d_i 是正还是负，也只影响这一条边。

再看一层：如果你把树从某个点 u 切开，分成“u 的子树”和“外面”，那么子树内部怎么分，会影响**外面**的只有两件事：

1. 这棵子树里有几个人去了甲县——因为甲县总共只能有 $n/2$ 个人，名额是要算账的；
2. u 自己去了哪个县——因为 u 和它父亲之间还有一条边，这条边算不算分，要看 u 在哪边。

除了这两件事，子树内部爱怎么折腾就怎么折腾，外面一点都不关心。

这句话是整道题的关键：**只要记住“子树里甲县有几个人”和“根在哪边”，子树内部的细节就可以全部打包丢掉**。

于是状态就定下来了。设

$f[u][j][k]$：只看 u 这棵子树，其中有 j 个人去了甲县，u 自己去了 k 县（$k=0$ 甲县，$k=1$ 乙县）时，子树内部能拿到的最大总分。

拿开头 $n=4$ 的例子把这张表填一小角，体会一下“细节丢掉”是什么意思。点 4 是叶子（$d_4=3$）：

- $f[4][1][0]=0$（4 去甲县），$f[4][0][1]=0$（4 去乙县），其余不合法；
- 点 2 先把孩子 4 并进来：比如“2 去乙县、4 去乙县”，父子同县加 $d_4=3$，得到 $f[2][0][1]=3$。注意这时表格里**只记了“子树里 0 个人去甲县、2 在乙县、最高 3 分”**，至于这 3 分具体是哪条边挣的，已经不用记了——对点 1 来说，知道这三项就够用了。

这就是“打包”：一张表替外面保管了子树里所有方案中最好的那些。

【做法三：把状态填出来】

单个点怎么初始化

u 单独一个人的时候，子树里只有它自己：

- 它去甲县：甲县人数是 1，所以 $f[u][1][0]=0$；
- 它去乙县：甲县人数是 0，所以 $f[u][0][1]=0$；
- 其他所有 $f[u][j][k]$ 都不可能，设成一个很小的数（后面叫它“不合法”）。

注意这里必须用“很小的数”而不是 0，因为答案是可能为负的，0 会被误当成一个合法方案。

怎么把一个孩子并进来

假设 u 已经处理完了前面几个孩子，现在来了一个新孩子 v（v 的整棵子树已经算好了）。

我们要把 v 的方案并到 u 的方案里。枚举四件事：

- u 这边已经分出 j 个人去甲县；
- u 在 k 县；
- v 那边分出 j_c 个人去甲县；
- v 在 k_c 县。

并起来以后：

- 甲县人数变成 $j + j_c$；
- u 的归属还是 k；
- 分数是 $f[u][j][k] + f[v][j_c][k_c]$，**并且如果 k 和 k_c 相同（父子同县），再加 d_v**。

加的是 d_v，也就是“孩子自己的编号”对应的那个数。这里最容易写错成 d_j（把变量名看混）或者 d_u（以为算的是父亲）。记住一句话：**每条边 u 到 v 的权值，用的是 v 的编号，不是 u 的**。

为什么必须“先算到临时数组，再整体抄回去”

有个坑必须单独讲。

合并前后，同一个格子 $f[u][j][k]$ 的含义变了：合并前 j 只统计“已经并进来的那些孩子”，合并后 j 要统计“连 v 一起”。也就是说，**同一个格子，合并前后装的不是同一件事**。

如果直接往 f 里取 $\max$，那么“旧含义”的值会赖在格子里不走，最后算出来的答案可能虚高。所以标准写法是：

1. 新开一个临时数组 g，里面只装“新含义”的结果，一开始全部设为不合法；
2. 每次算出 $f[u][j][k] + f[v][j_c][k_c] + \cdots$，更新到 g 上；
3. 所有情况枚举完，把 g 整体抄回 f。

这样旧值被彻底覆盖掉，一个都不会残留。

完整代码

```
#include <bits/stdc++.h>
using namespace std;

const int MAXN = 505;
const long long NEG = -1e18;     // 表示"这种情况不可能"。写成 -1e18，
                                 // 不要数 18 个零：多一个少一个都发现不了
```

```
int n;
long long d[MAXN];
vector<int> son[MAXN];      // son[u] 里存 u 的所有孩子

// f[u][j][k]: 只看 u 这棵子树，其中有 j 个人去了甲县，u 自己去了 k 县
//             (k=0 表示甲县，k=1 表示乙县）时，子树内部能拿到的最大均衡度
long long f[MAXN][MAXN][2];
long long g[MAXN][2];       // 合并孩子时先放这里的临时数组
int sz[MAXN];               // 子树里有几个点

void dfs(int u) {
    sz[u] = 1;
    for (int j = 0; j <= n; j++) {
        f[u][j][0] = NEG;
        f[u][j][1] = NEG;
    }
    f[u][1][0] = 0;         // u 去甲县，甲县人数 +1
    f[u][0][1] = 0;         // u 去乙县，甲县人数不变

    for (int t = 0; t < (int)son[u].size(); t++) {
        int v = son[u][t];
        dfs(v);

        // 先把临时数组清空
        for (int j = 0; j <= n; j++) {
            g[j][0] = NEG;
            g[j][1] = NEG;
        }
        // 枚举"u 这边已经分出 j 个人"和"v 那边分出 jc 个人"
        for (int j = 0; j <= sz[u]; j++) {
            for (int k = 0; k < 2; k++) {
                if (f[u][j][k] == NEG) {
                    continue;
                }
                for (int jc = 0; jc <= sz[v]; jc++) {
                    for (int kc = 0; kc < 2; kc++) {
                        if (f[v][jc][kc] == NEG) {
                            continue;
                        }
                        long long now = f[u][j][k] + f[v][jc][kc];
                        if (k == kc) {
                            now += d[v];          // 父子同县，这条边才算数
                        }
                        if (now > g[j + jc][k]) {
                            g[j + jc][k] = now;
                        }
                    }
                }
            }
        }
```

```
        sz[u] += sz[v];
        // 临时数组整体搬回 f，注意是"覆盖"，不是取更大
        for (int j = 0; j <= n; j++) {
            f[u][j][0] = g[j][0];
            f[u][j][1] = g[j][1];
        }
    }
}

int main() {
    scanf("%d", &n);
    for (int i = 1; i <= n; i++) {
        scanf("%lld", &d[i]);
    }
    for (int i = 2; i <= n; i++) {
        son[i / 2].push_back(i);      // i 的上级是 i/2
    }

    dfs(1);

    long long ans = f[1][n / 2][0];
    if (f[1][n / 2][1] > ans) {
        ans = f[1][n / 2][1];
    }
    printf("%lld\n", ans);
    return 0;
}
```

逐段看代码

- `NEG` 就是一个“很小的数”，表示“这种情况不可能”。加法和取 max 时它会自动被淘汰。注意代码里写的是 `-1e18`，而不是 18 个零——那种写法多数一个零、少数一个零，编译器不报错，答案全错，还很难查。`1e18` 是科学计数法字面量，10 的 22 次方以内的整幂都能被精确表示，直接当 `long long` 用没问题。以后写“无穷大”都照这个办：取一个比答案可能范围大两倍的数，用 `1e?` 写。
- `sz[u]` 是子树大小。后面所有循环的上界都卡在 `sz[u]` 和 `sz[v]` 上，这是复杂度的关键，下面单独讲。
- `for (int j = 0; j <= n; j++)` 那一段是把 u 的所有状态先清成不合法，再单独设置 $f[u][1][0]$ 和 $f[u][0][1]$。
- 中间的四层循环就是“枚举 u 这边的 (j,k)，再枚举 v 那边的 (j_c,k_c)”。
- `if (f[u][j][k] == NEG) continue;`：不合法的状态直接跳过。**这行很重要**，少了它，“不合法”这个值加上 d_v 之后会“复活”成一个看起来合法的数。
- `if (k == kc) now += d[v];`：父子同县才加分。
- 最后那段“整体抄回”就是上面说的第 3 步。

为什么复杂度是 $O(n^2)$

如果循环上界都写成 n，那合并一次是 n^2，一共 n 次合并，就是 $O(n^3)$。$n=450$ 时理论上要算九千万次，很悬。

所以要把上界卡到“当前的子树大小”：u 这边最多 `sz[u]` 个人，v 那边最多 `sz[v]` 个人。这样一来，一对点 (x,y) 只会在“它们的最近公共祖先”那里被配对一次，总转移量是 $O(n^2)$ 级别。$n=450$ 时几十万次，轻松通过。

有个细节要提醒：把上界写成 n 其实也**不一定**会超时，因为那些多出来的状态全是“不合法”，会被 `continue` 直接跳过。但这属于运气好，不能当成理由——写代码时把上界卡准，复杂度一眼就能看出来是 $O(n^2)$。

【新手容易踩的坑】

1. **用 `int` 存答案**。$|d_i|$ 可以到 10^9，n 到 450，答案能到 4.5×10^{11}，必须 `long long`。数据里专门有点卡这个。
2. **写回 max 而不是整体覆盖**。就是上面讲的那个坑，负权数据会直接暴露。
3. **加的权值搞错**。应该是 d_v（孩子的编号），不是 d_j、不是 d_u。
4. **忘了“正好 $n/2$ 人”**。最后要取 $f[1][n/2][0]$ 和 $f[1][n/2][1]$ 中较大的那个，不能取“至多 $n/2$”。
5. **叶子的初始化写错**。叶子只有 $f[u][1][0]=f[u][0][1]=0$ 这两项合法，其余都不合法。
6. **用“不合法”的值去参与加法**。要先判断 `== NEG` 再转移。
7. **常量里数零**。把负无穷写成一长串零（`-1` 后头跟 18 个 `0`），考场上手一抖多按一个，编译不报错、答案全错，还查不出来。统一写成 `-1e18`、`1e9 + 7`，一眼就能看出量级。

【这道题带走了什么】

- **先问一句“外面到底关心什么”**。这题外面只关心“子树里有几个人去甲县”和“根在哪边”，其它都能丢，状态就这么挤出来了。以后做树上题，先想这句话。
- **树上的选择问题，几乎都是“自底向上把子树打包”**。孩子先算完，再把孩子的结果合并进父亲，这种写法常被叫做“树上背包”，因为合并那一步长得像背包。
- **状态含义在合并前后变了，就必须换新数组覆盖**。这不只是这题的规定，是所有“两个集合合并”类 DP 的通病。